

LAPORAN HASIL PRAKTIKUM

UAS



Disusun Oleh:

Agrisyandi Maraja Hutabarat - Shan0n73 (254108020071)

Muhammad Daviq Naufal H. - lupocelin (254107020010)

Muhammad Hafiz - Fizzuz (254107020056)

Surya Sadikin Firdaus - l4utan (254107020105)

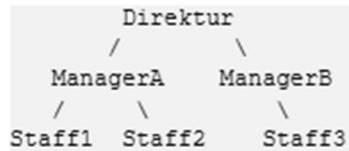
Valent Ridho Putra Santoso - Putraridho-cpu (254107020259)

Kelas 1H/TI

**PROGRAM STUDI D-IV TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026**

STUDI KASUS: STRUKTUR ORGANISASI PERUSAHAAN

Sebuah perusahaan memiliki struktur organisasi berbentuk binary tree. Setiap node merepresentasikan nama pegawai. Direktur berada di root, sedangkan bawahan langsung ditempatkan sebagai anak kiri dan anak kanan.



Tugas:

1. Buat program untuk menyimpan struktur organisasi tersebut menggunakan binary tree.
2. Tampilkan urutan pegawai menggunakan Preorder Traversal
3. Tampilkan urutan pegawai menggunakan Inorder Traversal.
4. Tampilkan urutan pegawai menggunakan Postorder Traversal.

Pertanyaan Analisis:

Jelaskan traversal mana yang paling cocok digunakan untuk menampilkan struktur dari atasan ke bawahan.

JAWABAN:

Tugas:

1. Main:

```
LupoCelin21, yesterday | 4 authors (Shan0n73 and others)
Shan0n73, yesterday * Base dengan contoh print relasi PnC d Hapus Nod...
LupoCelin21, yesterday | 4 authors (Shan0n73 and others)
public class Main {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        StrukturOrganisasi tree = new StrukturOrganisasi();

        // Base Tree dari Study Case
        tree.tambahPegawai(namaParent: null, namaBaru: "Direktur", posisi: ""); // (Root-nya)
        tree.tambahPegawai(namaParent: "Direktur", namaBaru: "ManagerA", posisi: "kiri");
        tree.tambahPegawai(namaParent: "Direktur", namaBaru: "ManagerB", posisi: "kanan");
        tree.tambahPegawai(namaParent: "ManagerA", namaBaru: "Staff1", posisi: "kiri");
        tree.tambahPegawai(namaParent: "ManagerA", namaBaru: "Staff2", posisi: "kanan");
        tree.tambahPegawai(namaParent: "ManagerB", namaBaru: "Staff3", posisi: "kiri");

        //Preorder
        tree.cetakPreOrder();

        // MENAMBAHKAN: Tampilkan urutan pegawai menggunakan Inorder Traversal
        tree.cetakInOrder();

        //Cetak postorder
        tree.cetakPostOrder();

        // Contoh: Print Parent 'n Child
        tree.cetakHubungan(nama: "ManagerA");
        tree.cetakHubungan(nama: "Staff3");

        // Contoh: Hapus Node
        tree.hapusPegawai(nama: "ManagerB");

    }
}
```

Node:

```
Shan0n73, yesterday | 1 author (Shan0n73)
1   Shan0n73, yesterday * Base dengan contoh pr
Shan0n73, yesterday | 1 author (Shan0n73)
2   public class Node {
3
4       String nama;
5       Node left, right;
6
7       public Node(String nama) {
8           this.nama = nama;
9           this.left = null;
10          this.right = null;
11      }
12  }
13
```

StrukturOrganisasi:

```
1 | LupoCelin21, yesterday | 4 authors (Shan0n73 and others)
  | Shan0n73, yesterday * Base dengan contoh print relasi PnC d Hapus Nod...
  | LupoCelin21, yesterday | 4 authors (Shan0n73 and others)
2 | public class StrukturOrganisasi {
3 |
4 |     Node root;
5 |
6 |     // Add Pegawai Baru
7 |     public boolean tambahPegawai(String namaParent, String namaBaru, String posisi) {
8 |         if (root == null) {
9 |             root = new Node(namaBaru);
10 |             System.out.println("Root telah dibuat: " + namaBaru);
11 |             return true;
12 |         }
13 |
14 |         Node parent = cariNode(root, namaParent);
15 |         if (parent == null) {
16 |             System.out.println("Error: Parent " + namaParent + " tidak ditemukan.");
17 |             return false;
18 |         }
19 |
20 |         if (posisi.equalsIgnoreCase(anotherString: "kiri")) {
21 |             if (parent.left == null) {
22 |                 parent.left = new Node(namaBaru);
23 |                 System.out.println("Menambahkan " + namaBaru + " sebagai anak kiri dari " + namaParent);
24 |                 return true;
25 |             } else {
26 |                 System.out.println("Error: Anak kiri dari " + namaParent + " sudah di-isi oleh " + parent.left.nama);
27 |             }
28 |         } else if (posisi.equalsIgnoreCase(anotherString: "kanan")) {
29 |             if (parent.right == null) {
30 |                 parent.right = new Node(namaBaru);
31 |                 System.out.println("Menambahkan " + namaBaru + " sebagai anak kanan dari " + namaParent);
32 |                 return true;
33 |             } else {
34 |                 System.out.println("Error: Anak kanan dari " + namaParent + " sudah di-isi oleh " + parent.right.nama);
35 |             }
36 |         } else {
37 |             System.out.println(x: "Error: Posisi harus 'kiri' atau 'kanan'.");
38 |         }
39 |         return false;
40 |     }
}
```

```

41
42 // Cari Node dengan Nama
43 private Node cariNode(Node current, String nama) {
44     if (current == null) {
45         return null;
46     }
47     if (current.nama.equalsIgnoreCase(nama)) {
48         return current;
49     }
50
51     Node foundLeft = cariNode(current.left, nama);
52     if (foundLeft != null) {
53         return foundLeft;
54     }
55
56     return cariNode(current.right, nama);
57 }
58
59 // Hapus Node + Subtree
60 public void hapusPegawai(String nama) {
61     if (root == null) {
62         System.out.println(x: "Tree kosong.");
63         return;
64     }
65     if (root.nama.equalsIgnoreCase(nama)) {
66         root = null;
67         System.out.println("Root '" + nama + "' dan seluruh cabangnya telah dihapus.");
68         return;
69     }
70
71     Node parent = cariParent(root, nama);
72     if (parent != null) {
73         if (parent.left != null && parent.left.nama.equalsIgnoreCase(nama)) {
74             parent.left = null;
75         } else if (parent.right != null && parent.right.nama.equalsIgnoreCase(nama)) {
76             parent.right = null;
77         }
78         System.out.println("\nPegawai '" + nama + "' beserta subtree-nya telah dihapus.");
79     } else {
80         System.out.println("Error: Pegawai '" + nama + "' tidak ada di tree.");
81     }
82 }

```

```

83
84 // Cari Parent dari Node
85 private Node cariParent(Node current, String namaAnak) {
86     if (current == null) {
87         return null;
88     }
89
90     if ((current.left != null && current.left.nama.equalsIgnoreCase(namaAnak))
91         || (current.right != null && current.right.nama.equalsIgnoreCase(namaAnak))) {
92         return current;
93     }
94
95     Node foundLeft = cariParent(current.left, namaAnak);
96     if (foundLeft != null) {
97         return foundLeft;
98     }
99
100     return cariParent(current.right, namaAnak);
101 }
102
103 // Print Parent 'n Child
104 public void cetakHubungan(String nama) {
105     Node target = cariNode(root, nama);
106     if (target == null) {
107         System.out.println("Pegawai '" + nama + "' tidak ditemukan.");
108         return;
109     }
110
111     System.out.println("\nRelasi dari " + target.nama + ":");
112
113     // Cari Parent
114     Node parent = cariParent(root, nama);
115     if (parent != null) {
116         System.out.println("Atasan (Parent) : " + parent.nama);
117     } else {
118         System.out.println(x: "Atasan (Parent) : Tidak ada (Ini adalah Direktur/Root)");
119     }
120
121     // Cari Child
122     System.out.println("Bawahan (Child Kiri) : " + (target.left != null ? target.left.nama : "-"));
123     System.out.println("Bawahan (Child Kanan) : " + (target.right != null ? target.right.nama : "-"));
124 }

```

```

125 ~ public void cetakPreOrder(){
126     System.out.println(x: "\nUrutan Pegawai (PreOrder Traversal):");
127     preOrderTraversal(root);
128     System.out.println();
129 }
130 ~ private void preOrderTraversal(Node current) {
131 ~     if (current != null) {
132         System.out.print(current.nama + " ");
133         preOrderTraversal(current.left);
134         preOrderTraversal(current.right);
135     }
136 }
137
138 // NO 3 : Print InOrder Traversal
139 ~ public void cetakInOrder() {
140     System.out.println(x: "\nUrutan Pegawai (InOrder Traversal):");
141     inOrderTraversal(root);
142     System.out.println();
143 }
144
145 // NO 3 : Fungsi rekursif InOrder (Kiri -> Root -> Kanan)
146 ~ private void inOrderTraversal(Node current) {
147 ~     if (current != null) {
148         inOrderTraversal(current.left); // Kunjungi anak kiri
149         System.out.print(current.nama + " "); // Cetak data parent/root
150         inOrderTraversal(current.right); // Kunjungi anak kanan
151     }
152 }
153
154 //Print Postorder Traversal
155 ~ public void cetakPostOrder() {
156     System.out.println(x: "\nUrutan Pegawai (Postorder Traversal):");
157     postOrder(root);
158     System.out.println();
159 }
160
161 // Fungsi utama rekursif Postorder (Kiri -> Kanan -> Root)
162 ~ private void postOrder(Node node) {
163 ~     if (node == null) {
164         return;
165     }
166 }

```

```

162 // Fungsi utama rekursif Postorder (Kiri -> Kanan -> Root)
163 private void postOrder(Node node) {
164     if (node == null) {
165         return;
166     }
167     postOrder(node.left); // 1. Telusuri anak kiri terlebih dahulu
168     postOrder(node.right); // 2. Telusuri anak kanan kedua
169     System.out.print(node.nama + " "); // 3. Cetak nama pegawai terakhir
170 }
171
172 }
173
174

```

```

130 private void preOrderTraversal(Node current) {
131     if (current != null) {
132         System.out.print(current.nama + " ");
133         preOrderTraversal(current.left);
134         preOrderTraversal(current.right);
135     }
136 }
137

```

2.

3.

```
144
145 // NO 3 : Fungsi rekursif InOrder (Kiri -> Root -> Kanan)
146 private void inOrderTraversal(Node current) {
147     if (current != null) {
148         inOrderTraversal(current.left); // Kunjungi anak kiri
149         System.out.print(current.nama + " "); // Cetak data parent/root
150         inOrderTraversal(current.right); // Kunjungi anak kanan
151     }
152 }
153
154
```

4.

```
162 // Fungsi utama rekursif Postorder (Kiri -> Kanan -> Root)
163 private void postOrder(Node node) {
164     if (node == null) {
165         return;
166     }
167     postOrder(node.left); // 1. Telusuri anak kiri terlebih dahulu
168     postOrder(node.right); // 2. Telusuri anak kanan kedua
169     System.out.print(node.nama + " "); // 3. Cetak nama pegawai terakhir
170 }
171
172
173 }
174
```

Pertanyaan Analisis:

PreOrder, alasannya karena Preorder memproses node induk sebelum anak-anaknya, sehingga hierarki atasan selalu muncul lebih dulu daripada bawahannya — persis seperti struktur organisasi dari Direktur → Manajer → Staf.